

Dynamic Memory Allocation and Virtual Memory Management

This briefing document provides a comprehensive overview of dynamic memory allocation and virtual memory management in computer systems, drawing from the provided sources. It covers fundamental concepts, various allocation schemes, fragmentation issues, and advanced paging techniques, including their implementation and trade-offs.

1. Dynamic Memory Allocation: Fundamentals and Challenges

Dynamic memory allocation is a critical aspect of computing, enabling systems to allocate memory for transient objects as needed and free it when no longer required. This "dynamic" nature introduces complexities, primarily in the form of fragmentation.

1.1 When Memory is Allocated

Memory allocation occurs throughout computing systems, from user-level programs to the operating system kernel itself:

- **User-Level Allocations:**

- **C/C++ Heap:** When objects are created and destructed in C++ using `new` and `delete`, or in C using `malloc()` and `free()`, memory allocation functions are invoked on the process heap. The `malloc()` function, in particular, is a library call, not a system call, and manages space within a process's virtual address space, building on kernel system calls like `brk` or `mmap()` to request or release memory from the OS.

- **Java Virtual Machine & Databases:** These systems feature sophisticated memory managers, often with garbage collection in Java's case, to optimize memory usage.

- **Kernel-Level Allocations (on user's behalf):** The operating system kernel allocates memory for user processes in several situations:

- **Process Creation:** When a new process is created (e.g., using `fork()`).
- **Program Loading:** When a new program is loaded into memory (e.g., using `execve()`).
- **Process Stack Growth:** If a process's stack grows beyond its initially allocated size, the OS adds more memory. (Note: User-level thread stacks are typically allocated by the user at thread creation time and grow from the process heap).
- **Heap Expansion:** When a user program requests a large amount of heap memory, the kernel may provide additional memory to expand the heap.
- **Shared Memory Segments:** Processes can explicitly create memory segments sharable across processes, which are provided by the kernel.
- **Memory Mapping Files/Devices:** The `mmap()` command allows processes to map files or devices directly into memory, requiring the OS to allocate memory for these mappings.
- **Kernel's Own Memory Management:** The operating system also dynamically manages its own memory for kernel objects. This is often handled by specialized object-caching allocators like the **Slab Allocator**.

1.2 Fragmentation

Fragmentation is a significant problem in dynamic memory allocation, preventing efficient use of available memory.

- **External Fragmentation:** "It occurs when we need to allocate a contiguous segment of memory but cannot find one that is sufficiently large. In this particular case, none of the available segments is big enough to fit the new request. We do have enough memory just no available

segment that is big enough." This is difficult to resolve because moving allocated segments can invalidate existing memory references (addresses).

- **Internal Fragmentation:** This occurs when memory is divided into fixed-size allocatable units (like blocks or pages), and a requested allocation is smaller than the unit size. The unused portion within the allocated unit is "wasted." It also occurs when a segment is allocated a contiguous sequence of blocks, and the end of the segment does not perfectly align with the end of the last block.

1.3 Memory Allocation Data Structures and Approaches

To manage memory, systems typically use data structures to track allocated and available (free) memory.

- **Linked Lists for Allocated and Free Segments:**

- Allocated segments are often tracked in a linked list, with each node pointing to the start and end of a segment.

- Available memory segments are maintained in a "free list," also commonly a linked list.

- **Naive Free-List Allocator:**

- In a simple linked free-list implementation, a request for memory involves traversing the list to find a suitable segment. This can be very slow for large memory spaces, as "we don't know this until we have traversed the entire free list." Improvements can include maintaining a maximum available segment size to quickly reject overly large requests.

- **Heuristics for Allocation:** When multiple free segments fit a request, various heuristics can be used: * **First Fit:** Uses the "first available segment that fits." * **Best Fit:** Finds "the smallest available segment that fits." This is more expensive than First Fit due to the need to compare all fitting segments. It "may give rise to lots of small available segments." * **Worst Fit:** Picks "the largest available segment that fits." This "keeps the sizes of the available segments about equal."

- **Buddy System Allocation:**

- A more sophisticated and widely used scheme, particularly in kernels (e.g., Linux for frame/page allocation).

- It addresses the inefficiency of a single free list by maintaining "an entire collection of free lists, with one free list for small available segments and then other free lists for increasingly larger available segments."

- Segment sizes are typically restricted to power-of-two multiples of a basic size.

- **Allocation:** Requests are rounded up to the next power-of-two size. If a segment of that size is available, it's allocated. Otherwise, a larger segment is split into two "buddies," one is used, and the other is returned to the appropriate free list. This is a recursive process.

- **De-allocation:** A freed segment is returned to its free list. If its "buddy" is also free, they are "coalesced" (glued together) into a larger segment and returned to the next larger free list. This coalescing can also recursively cascade.

- The Buddy System effectively manages external fragmentation by restricting segment sizes and enabling coalescing. However, it can still suffer from internal fragmentation if a requested size is significantly smaller than the next power-of-two block size.

- Invented by Harry Markowitz (Nobel laureate in Economics).

- **Slab Allocator:**

- Used in kernels (e.g., Linux for finer granularity allocations) for "object-caching memory allocators."

- Optimized for "memory allocation for objects that have one of very few different sizes."

- Beneficial for managing kernel objects (e.g., file descriptors, thread control blocks) by caching object structures to save on constructor/destructor overhead.

1.4 Memory Allocation in Linux

- `alloc_pages()` and `__get_free_pages()`: These low-level kernel functions allocate contiguous sequences of physical memory "frames" or "pages." They primarily use the **Buddy System**.
- `kmalloc()`: Allocates a physically contiguous sequence of bytes in physical memory.
- `vmalloc()`: Allocates a virtually contiguous sequence of bytes in virtual memory.
- **Slab Allocator**: Used for finer granularity allocations within the kernel, optimized for fixed-size objects. Linux uses variations like SLAB, SLUB, and SLOB.
- `malloc()` (**User-level**): Implemented as a user-level library call, not a direct system call. It manages a process's heap within its virtual address space.

2. Virtual Memory and Address Spaces

Virtual memory provides a powerful abstraction that greatly simplifies programming and enhances system efficiency.

2.1 The Illusion of Virtual Memory

- **Programmer's Perspective**: "every address generated by a user program is a virtual address. The OS is just providing an illusion to each process, specifically that it has its own large and private memory."
- **Benefits**:
 - **Ease of Use**: Programs perceive a large, contiguous address space, eliminating concerns about physical memory limitations or where to store variables.
 - **Isolation and Protection**: Prevents one program from accessing or overwriting another program's memory.
 - **Larger-than-Physical Memory Processes**: Allows processes to be larger than the available physical memory.
 - **Efficient Multiprogramming**: By only loading necessary parts of processes into physical memory, more processes can reside in memory concurrently, leading to better CPU utilization.
- **Physical vs. Virtual Addresses**:
 - **Physical Memory/Address Space**: The actual memory hardware of the machine, with addresses starting from 0.
 - **Logical/Virtual Address Space**: The address space perceived by a process, typically starting at address 0 and extending to a large (potentially arbitrary) size. Addresses within this space are called "logical" or "virtual" addresses.
 - **Address Resolution**: Hardware (Memory Management Unit - MMU) and the OS work together to translate virtual addresses into unique physical addresses.

2.2 Process Memory Layout

A typical process's virtual address space is organized into distinct areas:

- **Executable Code**: Stored near the beginning of the address space.
- **Stack**: Located at the opposite end of the address space, where function arguments, return addresses, and local variables are stored. It grows and shrinks dynamically.

- **Heap:** Where dynamically allocated data (e.g., via `malloc()` or `new`) is stored. It can grow as memory is requested.
- **Free Space:** A significant portion of the address space is often unused, allowing for sparse address spaces. Modern 64-bit processors support extremely large address spaces (up to 2^{64} bytes), far exceeding typical program needs.

3. Paging: Eliminating External Fragmentation

Paging is a core mechanism for virtual memory management, addressing the external fragmentation issues inherent in simpler relocation schemes.

3.1 Paging Concept

- **Fixed-Size Partitions:** Instead of relocating entire memory regions contiguously, paging partitions the logical address space into "small, equal-sized portions which we call pages." Similarly, physical memory is divided into fixed-size "frames" of the same size.
- **Individual Relocation:** Each logical page is "individually relocated to a portion in physical memory," meaning pages of a logical memory region "do not need to be contiguous or even adjacent" in physical memory.
- **Elimination of External Fragmentation:** By breaking memory into fixed-size units, "When the units to allocate all have the same size, external fragmentation stops being an issue."
- **Internal Fragmentation in Paging:** Paging, however, still suffers from internal fragmentation because "we are most likely not filling the last page used to store a memory region." On average, "we waste half of the last frame for each memory region."
- **Page Size Trade-off:**
 - **Large Frames:** Lead to more internal fragmentation (larger wasted space in the last page) but less overhead (smaller page tables, fewer pages to swap).
 - **Small Frames:** Lead to less internal fragmentation but more overhead (larger page tables, more expensive disk I/O for swapping many small pages). "The system designer now needs to choose a page or frame size that balances internal fragmentation against performance."

3.2 Paging in Hardware: Address Resolution

The Memory Management Unit (MMU) handles the translation of logical addresses to physical addresses:

1. **CPU Issues Logical Address:** The CPU generates a logical address.
2. **Split Address:** The logical address is split into two parts:
 - **Page Number:** Identifies the logical page.
 - **Offset:** Identifies the location within that page.
3. **Page Table Lookup:** The "page number is used to index into a so-called page table." The page table is an array where the index is the page number.
4. **Retrieve Frame Number:** The entry at the given page number in the page table contains "the number of the frame where the given page is stored in physical memory."
5. **Construct Physical Address:** The "frame number that we just looked up... and the offset that we get when we split the logical address" are glued together. The offset is used without modification because the offset within a page is the same as the offset within a frame.
6. **Access Physical Memory:** The resulting physical address is used to reference the location in physical memory.

This "address resolution" process is repeated for every logical address issued by the CPU.

4. Advanced Paging: Page Table Implementations

For large address spaces and multiple processes, naive page table implementations become unmanageably large, requiring advanced techniques to reduce their memory footprint and improve performance.

4.1 Challenges with Naive Page Tables

- **Size:** A simple "one big array" page table in memory is problematic. For example, a 32-bit address space with 4KB pages and 4-byte entries results in "about 1 million entries... up to 4MB per page table." For each process, this consumes a large amount of memory. A 64-bit address space would require a "humongous" page table.

- **Wasted Entries:** "Most of the address space of the process is not used," meaning many page table entries would be empty and "wasted."

4.2 Multi-Level (Hierarchical) Paging

This widely adopted solution (e.g., x86 architecture, SPARC, Motorola 68030, Linux) addresses the size issue by "paging the page table" itself.

- **Concept:**

1. The large linear page table is partitioned into "page table pages," each fitting within a single memory page.

2. A "page directory" (or higher-level directories) is introduced to track these page table pages. Each entry in the page directory points to a page table page.

3. If a large portion of the logical address space is unused, the corresponding "page table pages" and their entries in the page directory simply "would simply be invalid, with no page table page allocated."

4. This means "The amount of memory needed to store the page table is now a function of how much of the address space is used rather than how big the address space is."

- **Address Resolution (Two-Level Example):**

1. The logical address is split into: a **page table number** (or page directory index), a **page number** (or page table index), and an **offset**.

2. The **page table number** is used to index into the page directory to find the physical location of the relevant page table page.

3. Once the page table page is located, the **page number** is used to index into it to find the frame number.

4. The frame number and offset are concatenated to form the physical address.

- **Advantages:**

- **Space Savings:** Significantly reduces memory consumption for page tables, especially for sparse address spaces.

- **Flexible Memory Management:** Each portion of the page table fits neatly within a page, allowing the OS to allocate them anywhere in physical memory.

- **Disadvantages:**

- **Performance Overhead:** On a TLB miss, resolving a logical address in a multi-level table requires "several memory references" (one for each level of the directory, plus one for the actual page table entry) before the physical address can be constructed. This can "slow down the process by a factor of two or more." Caching (e.g., using a Translation Lookaside Buffer - TLB) is crucial to mitigate this.

- **Complexity:** The lookup process is more intricate.

- **Multiple Levels:** The number of levels can vary depending on the address space size (e.g., 32-bit x86 uses two levels, SPARC uses three, some 64-bit architectures use five or six levels).

4.3 Inverted Page Tables

An alternative approach to keeping page tables small.

- **Concept:** Instead of a separate page table per process, there is "only one page table in the system." This table is "indexed by the frame number rather than by the page number."
- **Structure:** It's an array of page entries where each entry corresponds to a physical frame. Each entry specifies "which process is using this page, and which virtual page of that process maps to this physical page." To distinguish processes, logical addresses are prefixed with a process identifier.
- **Advantages:**
 - **Space Savings:** The page table size "grows with the amount of physical memory in the system," not with the size of the virtual address space or the number of processes.
 - **Single Table:** Only one table for the entire system to manage.
- **Disadvantages:**
 - **Search Time:** Resolving an address requires "searching through the page table to find an entry" that matches the process ID and page number. A linear search is "obviously poor."
 - Historically used in early virtual memory systems (e.g., Atlas computer).

4.4 Hashed Page Tables

An improvement on inverted page tables to address the search time issue.

- **Concept:** Uses a hash function to map the combination of process ID and page number to an index in the page table.
- **Structure:** The page table is augmented with columns for process ID and page number. Instead of direct indexing, the hash of (process ID + page number) is used to index into the table.
- **Advantages:**
 - **Performance:** Significantly improves lookup speed compared to a linear search.
 - **Space Savings:** Similar to inverted page tables, it scales with physical memory size.
 - **Single Table:** A single page table for the system (if process ID is part of the logical address).
- **Disadvantages:**
 - **Collisions:** Multiple (process ID + page number) combinations can hash to the same entry. These are handled using techniques like chaining.
- **Popularity:** Used in many modern 64-bit architectures (e.g., IBM Power, HP PA-RISC, Intel Itanium).

5. Memory Management in Operating Systems

Operating systems implement various policies and algorithms to manage virtual memory effectively.

5.1 Virtual Memory Terminology

- **Virtual Memory:** "A storage allocation scheme in which secondary memory can be addressed as though it were part of main memory."
- **Virtual Address:** "The address assigned to a location in virtual memory to allow that location to be accessed as though it were part of main memory."
- **Virtual Address Space:** "The virtual storage assigned to a process."

- **Address Space:** "The range of memory addresses available to a process."
- **Real Address:** "The address of a storage location in main memory."
- **Page Fault:** An interrupt generated by the processor when it encounters a logical address that is not currently in main memory. This triggers the OS to load the needed page.
- **Resident Set:** "The portion of a process that is actually in main memory at any time."
- **Thrashing:** A condition where the system "spends most of its time swapping pieces rather than executing instructions" due to too many page faults.
- **Locality:** The principle that program and data references within a process tend to cluster, making virtual memory schemes practical.
- **Translation Lookaside Buffer (TLB):** A "special high-speed cache for page table entries" to speed up address translation by storing recently used page table entries.

5.2 OS Policies for Virtual Memory

The OS makes design choices regarding:

- **Fetch Policy:** Determines when a page is brought into main memory.
 - **Demand Paging:** Page is brought in "only when a reference is made to a location on that page." Most common.
 - **Prepaging:** Brings in "pages other than the one demanded by a page fault." Exploits disk characteristics to transfer contiguous blocks efficiently.
- **Placement Policy:** Where in real memory a process piece resides. For paging, this is usually irrelevant as pages can go anywhere. It's more relevant for segmentation or NUMA systems.
- **Replacement Policy:** Which page in main memory is replaced when a new page needs to be brought in. Aims to remove the page least likely to be referenced soon.
 - **Optimal:** Replaces the page that will be referenced furthest in the future (impossible to implement, but a benchmark).
 - **Least Recently Used (LRU):** Replaces the page "that has not been referenced for the longest time." Difficult to implement efficiently.
 - **First-In-First-Out (FIFO):** Treats page frames as a circular buffer, replacing the oldest page. Simple but can perform poorly by repeatedly paging out heavily used pages.
 - **Clock Policy (Second-Chance Algorithm):** A variant of FIFO that uses a "use bit." If a page's use bit is 1 (recently accessed), it's reset to 0, and the pointer moves on. If 0, the page is replaced. More sophisticated variants incorporate a "modify bit" (u=0, m=0 preferred for replacement) to save write-back time.
 - **Page Buffering:** Improves performance by not immediately discarding replaced pages but placing them on a "free page list" (unmodified) or "modified page list" (dirty). This acts as a cache and allows batch writes for modified pages.
- **Resident Set Management:** How many page frames are allocated to each active process.
 - **Fixed Allocation:** A process gets a fixed number of frames. Replacement occurs among its own pages (local scope).
 - **Variable Allocation:** The number of frames allocated to a process can change over time.
 - * **Global Scope:** Replaced pages can be chosen from any unlocked page in main memory, regardless of which process owns it. Simple, but can reduce the resident set of an "innocent" process.
 - * **Local Scope:** Replacement is limited to the pages of the faulting process.
 - **Working Set Strategy:** Aims to keep a process's "working set" (pages recently referenced within a time window Δ) in memory. Difficult to implement perfectly, but approximated by

monitoring page fault rates (e.g., Page Fault Frequency - PFF, Variable-Interval Sampled Working Set - VSWS).

- **Cleaning Policy:** When a modified (dirty) page is written back to secondary memory.
 - **Demand Cleaning:** Page is written out only when selected for replacement.
 - **Precleaning:** Writes modified pages before they are needed, potentially in batches, but can waste I/O if pages are modified again before replacement.
 - Page buffering can decouple cleaning from replacement.
- **Load Control:** Determines the "multiprogramming level" (number of processes in main memory). Too few leads to wasted CPU time; too many leads to thrashing. Algorithms like working set or PFF implicitly manage this, or policies like L=S criterion (mean time between faults = mean time to process a fault).

5.3 Memory Management in Specific Operating Systems

- **UNIX and Solaris:**
 - Use paged virtual memory.
 - Maintain per-process page tables, disk block descriptors (for disk copy info), page frame data tables (for real memory frames), and swap-use tables.
 - Page replacement uses a "two-handed clock algorithm," a refinement of the clock policy with a fronthand (clears reference bits) and a backhand (checks and replaces pages with cleared reference bits).
 - Kernel memory allocation uses a modified **Buddy System** ("lazy buddy system") for small objects, deferring coalescing to reduce overhead.
- **Linux:**
 - Uses a three-level page table structure conceptually (page directory, page middle directory, page table), though optimized at compile time for architectures like x86 that only support two levels.
 - Page frames are managed using a **Buddy System** for contiguous blocks.
 - Older page replacement used an 8-bit age variable (least frequently used policy).
 - Newer (2.6.28+) "split LRU algorithm" uses `PG_active` and `PG_referenced` flags and active/inactive lists in memory zones, with a `kswapd` daemon for periodic reclamation.
 - Kernel memory allocation for small chunks uses **Slab Allocation** (SLAB, SLUB, SLOB), built on top of the buddy system.
- **Windows:**
 - Virtual memory manager controls allocation and paging.
 - On 32-bit platforms, processes see a 4GB virtual address space (2GB for user, 2GB for OS). 64-bit platforms offer 8 TB for user.
 - Pages are fixed-size (e.g., 4KB on Intel, 8KB on Itanium).
 - Employs a **variable allocation, local scope** resident set management scheme.
 - Uses both `pagefile.sys` for general paging and `swapfile.sys` (for Windows Store apps) for swapping entire processes.